Unit – II

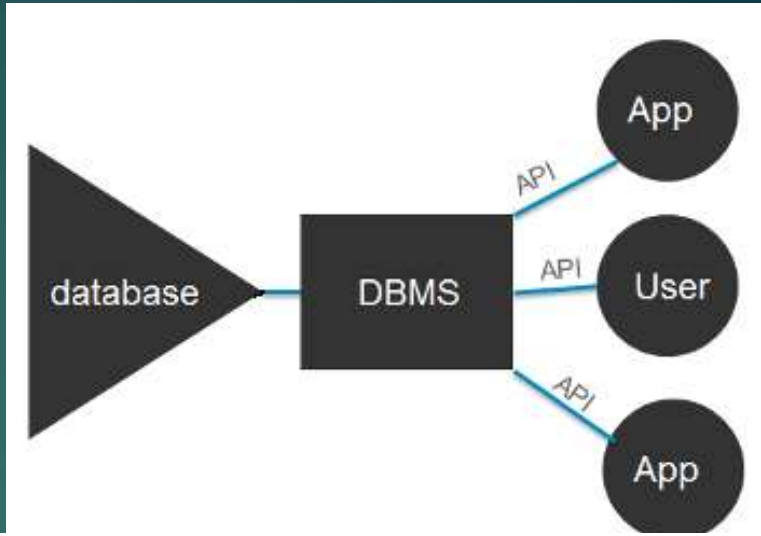
Java DataBase Connectivity (JDBC)

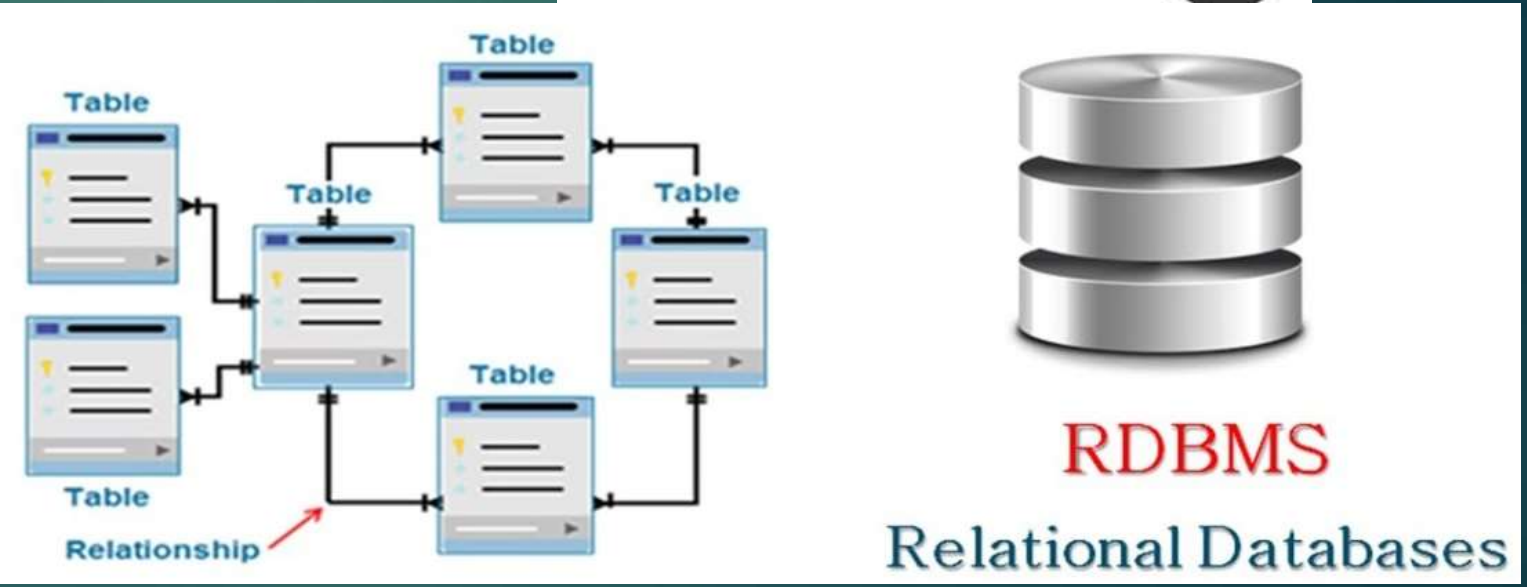
Unit – II

Java DataBase Connectivity (JDBC)

- 3.1 Two-Tier Database Design, Three Tier Database Design
- 3.2 The JDBC API: The API components, database operations like creating tables, CRUD(Create, Read, Update, Delete) operations using SQL
- 3.3 JDBC- advantages and disadvantages
- 3.4 JDBC drivers
- 3.5 JDBC-ODBC bridge
- 3.6 Develop java program using JDBC

Introduction





CLIENT-SERVER DESIGN:

- Two-tier database design
- Three-tier database design

JDBC

- JDBC stands for Java Database Connectivity.
- JDBC was developed by JavaSoft of Sun microsystems.
- JDBC is a standard Java API for database independent connectivity between the Java programming language and a wide range of databases.
- It defines interfaces and classes for writing database applications in Java by making database connections.
- Using JDBC you can send SQL, PL/SQL statements to almost any relational database.

JDBC

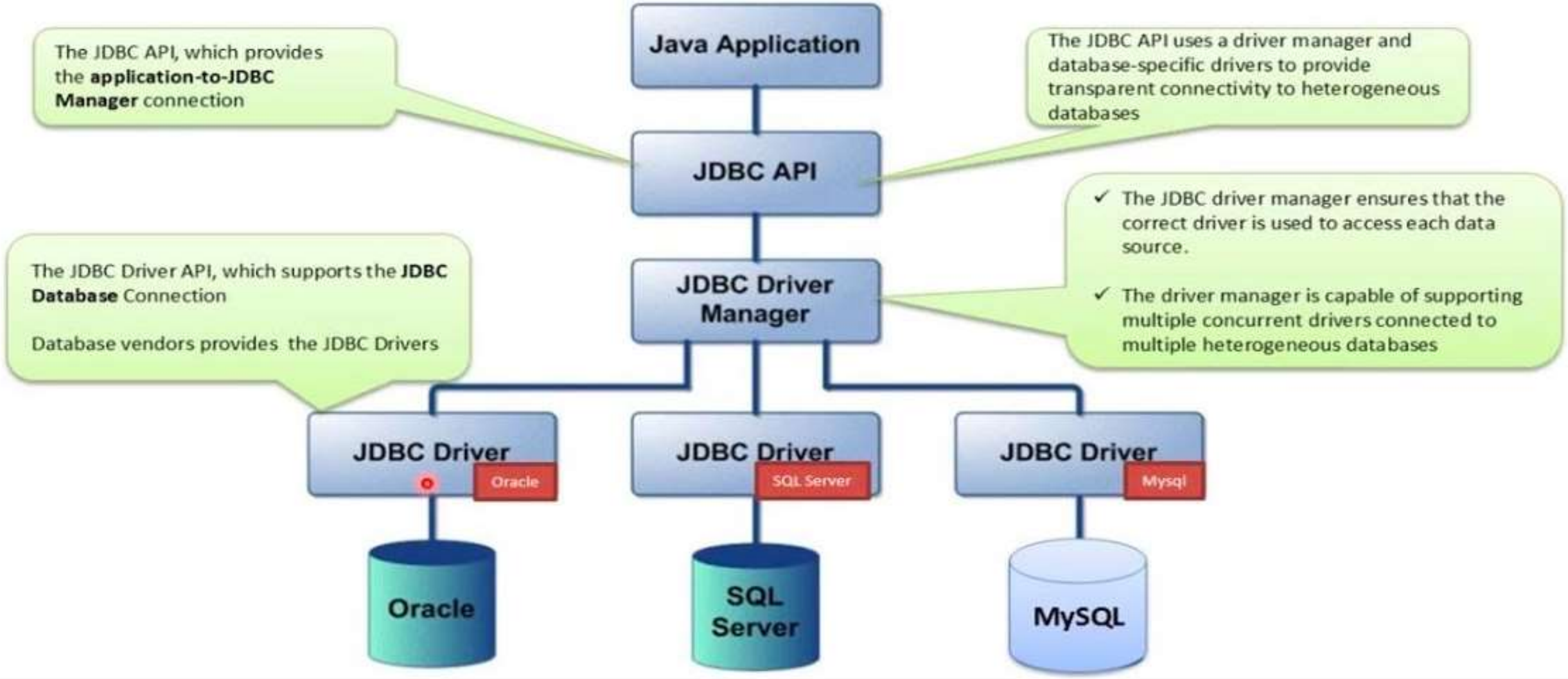
- JDBC is a Java API for executing SQL statements and supports basic SQL functionality.
- It provides RDBMS access by allowing you to embed SQL inside Java code.
- Since nearly all relational database management systems (RDBMSs) support SQL, and because Java itself runs on most platforms, JDBC makes it possible to write a single database application that can run on different platforms and interact with different DBMSs.

JDBC

- Java Database Connectivity is similar to Open Database Connectivity (ODBC) which is used for accessing and managing database, but the difference is that JDBC is designed specifically for Java programs, whereas ODBC is not depended upon any language.
- In short JDBC helps the programmers to write java applications that manage these three programming activities:

1. Connect to a data source, like a database.
2. Send queries and update statements to the database.
3. Retrieve and process the results received from the database in answer to your query.

JDBC Architecture



JDBC Architecture

- The JDBC API supports both two-tier and three-tier processing models for database access.

General JDBC Architecture

- The JDBC API supports both two-tier and three-tier processing models for database access but in general JDBC Architecture consists of two layers:

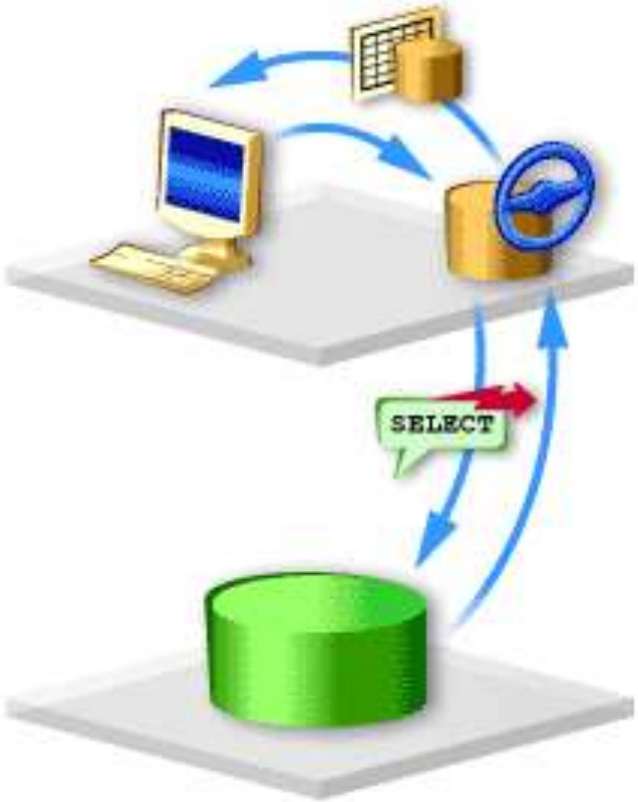
1. JDBC API: This provides the application-to JDBC Manager connection.
2. JDBC Driver API: This supports the JDBC Manager-to-Driver Connection.

JDBC Architecture

- The JDBC API uses a driver manager and database specific drivers to provide transparent connectivity to heterogeneous databases.
- The JDBC driver manager ensures that the correct driver is used to access each data source.
- The driver manager is capable of supporting multiple concurrent drivers connected to multiple heterogeneous databases.

Two-Tier Database Design

Client Tier
The client is the first tier in the two-tier model.



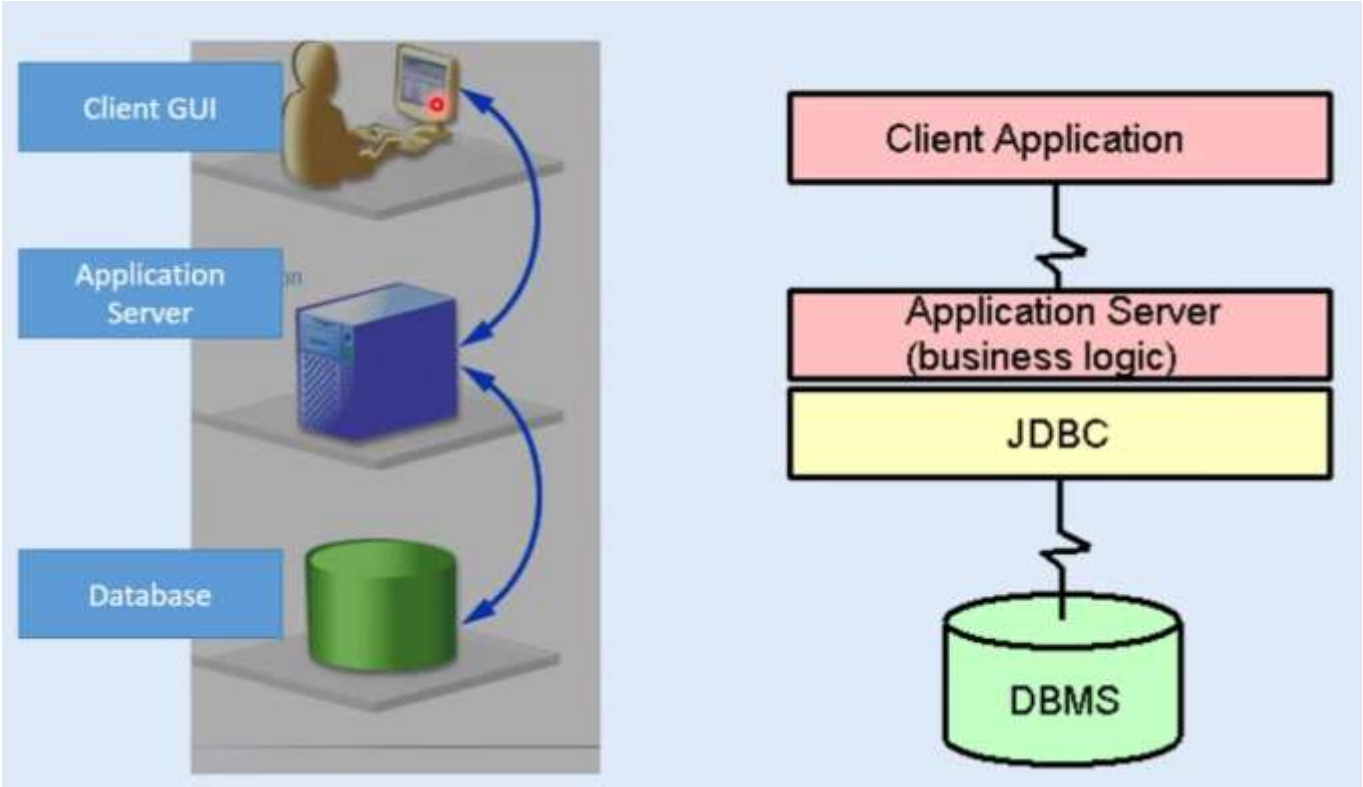
JDBC Driver
Provides the request/response communication between the two tiers.

DBMS Tier
This is the persistent datastore (e.g., a database) which may reside on the same computer as the client.

Two-Tier Database Design

- In the two-tier model, a java application talks directly to the data source.
- This requires a JDBC driver that can communicate with the particular data source being accessed.
- A user's commands are delivered to the database or other data source, and the results of those statements are sent back to the user.
- The data source may be located on another machine to which the user is connected via a network.
- This is referred to as a client/server configuration, with the user's machine as the client, and the machine housing the data source as the server.
- The network can be an intranet, which, for example, connects employees within a corporation, or it can be the internet.

Three-Tier Database Design



Three-Tier Database Design

- In the three-tier model, commands are sent to a "middle tier" of services, which then sends the commands to the data source.
- The data source processes the commands and sends the results back to the middle tier, which then sends them to the user.
- MIS (management information system)directors find the three-tier model very attractive because the middle tier makes it possible to maintain control over access and the kinds of updates that can be made to corporate data.
- Another advantage is that it simplifies the deployment of applications.
- Finally, in many cases, the three-tier architecture can provide performance advantages.

Three-Tier Database Design

- With enterprises increasingly using the Java programming language for writing server code, the JDBC API is being used more and more in the middle tier of a three-tier architecture.
- Some of the features that make JDBC a server technology are its support for connection pooling, distributed transactions, and disconnected rowsets.
- The JDBC API is also what allows access to a data source from a Java middle tier.

3.2 The JDBC API

THE API COMPONENTS, DATABASE OPERATIONS LIKE CRUD(CREATE, READ, UPDATE, DELETE) OPERATIONS USING SQL

JDBC PRODUCT COMPONENTS

- JDBC has four Components:
  1. The JDBC API.
  2. The JDBC Driver Manager.
  3. The JDBC Test Suite.
  4. The JDBC-ODBC Bridge.

1. JDBC API

- JDBC API (application programming interface) is a part of the Java platform that have included in Java Standard Edition (Java SE) and the Java Enterprise Edition (Java EE) in itself.
- The latest version of application programming interface is divided into two packages
  1. java.sql
  2. javax.sql
- These packages contain all the APIs which provide programmatic access to a relational database (like oracle, SQL server, MySQL etc.)
- By using it, front end java applications can execute query and fetch data.

2. JDBC DRIVER MANAGER

- This interface manages a list of database drivers, matches connection requests from the java application with the proper database driver using communication sub protocol.
- The first driver that recognizes a certain sub protocol under JDBC will be used to establish a database connection.
- Internally, JDBC Driver Manager is a class in JDBC API. The objects of this class can connect Java applications to a JDBC driver.
- Driver Manager is the very important part of the JDBC architecture. The main responsibility of JDBC Driver Manager is to load all the drivers found in the system properly.
- The Driver Manager also helps to select the most appropriate driver from the previously loaded drivers when a new open database is connected.

3. JDBC TEST SUITE

- The function of JDBC driver test suite is to make ensure that the JDBC drivers will run user's program or not.
- The test suite of JDBC application program interface is very useful for testing a driver based on JDBC technology during testing period.
- JDBC Test suite facilitates the programmer to test the various operations such as deletion, updation, insertion that are being executed by the JDBC Drivers.
- It is used to check compatibility of JDBC driver with (J2EE) Java Platform Enterprise Edition.

4. JDBC-ODBC BRIDGE

- The JDBC-ODBC Bridge, also known as JDBC type 1 driver is a database driver that utilizes the ODBC driver to connect the database.
- JDBC-ODBC Bridge Drivers are used to connect the database drivers to the database.
- This driver translates JDBC method calls into ODBC function calls. The Bridge implements JDBC for any database for which an ODBC driver is available. The Bridge is always implemented as the sun.jdbc.odbc Java package and it contains a native library used to access ODBC.
- ODBC driver is already installed or come as default driver in windows.

JDBC API

JDBC API

- The JDBC API provides the facility for accessing the relational database from the Java programming language.
- The user not only execute the SQL statements, retrieve results, and update the data but can also access it anywhere within a network because of its "Write Once, Run Anywhere" (WORA) capabilities.
- The JDBC API can also interact with multiple data sources in a distributed environment.
- Due to JDBC API technology, user can also access other tabular data sources like spreadsheets or flat files even in the heterogeneous environment.

JDBC API

- The JDBC API provides the following interfaces and classes:
- **Driver Manager**: This class manages a list of database drivers. Matches connection requests from the java application with the proper database driver using communication sub protocol. The first driver that recognizes a certain sub protocol under JDBC will be used to establish a database Connection.
- **Driver**: This interface handles the communications with the database server. You will interact directly with Driver objects very rarely. Instead, you use a DriverManager object, which manages objects of this type. It also abstracts the details associated with working with Driver objects

JDBC API

- **Connection**: This interface with all methods for contacting a database. The connection object represents communication context, i.e., all communication with database is through connection object only.
- **Statement**: You use objects created from this interface to submit the SQL statements to the database. Some derived interfaces accept parameters in addition to executing stored procedures.
- **ResultSet**: These objects hold data retrieved from a database after you execute an SQL query using Statement objects. It acts as an iterate to allow you to move through its data.
- **SQLException**: This class handles any errors that occur in a database application.

IMPORTANT CLASSES AND INTERFACES

- The mechanism to communicate with database server exists as standard java classes.
- JDBC API has abstract java interfaces that allow to link to any number of databases.
- **java.sql.DriverManager**: handles the loading and unloading of appropriate database drivers required to make the connection.
- **java.sql.Connection**: exposes the database to the developer and give connection as a java component.
- **java.sql.Statement**: provides a container for executing SQL statement using connection.
- **java.sql.ResultSet**: represent the data that returns form the database server to java application.

DRIVER MANAGER CLASS

- The DriverManager class acts as an interface between user and drivers.
- It keeps track of the drivers that are available and handles establishing a connection between a database and the appropriate driver.
- The DriverManager class maintains a list of Driver classes that have registered themselves by calling the method DriverManager.registerDriver().

DRIVER MANAGER CLASS

Sr	Methods
1.	`public static void registerDriver(Driver driver):`
	is used to register the given driver with DriverManager.
2.	`public static void deregisterDriver(Driver driver):`
	is used to deregister the given driver (drop the driver from the list) with DriverManager.
3.	`public static Connection getConnection(String url):`
	is used to establish the connection with the specified url.

DRIVER MANAGER CLASS

Sr	Methods
4.	`getConnection(String url,String userName,String password):`
	is used to establish the connection with the specified url, username and password.
5.	`getDrivers()`
	Access a list of drivers present in database
6.	`println()`
	Prints a message used in log stream

Sr	Methods
7.	`setLoginTimeout(int seconds):`
	Sets the maximum time in seconds that the DriverManager will wait for a connection to be established.
8.	`getLoginTimeout()`
	Get the maximum time in seconds that the DriverManager will wait for a connection to be established.

DRIVER MANAGER CLASS

RESULTSET INTERFACE

- The object of ResultSet maintains a cursor pointing to a particular row of data. Initially, cursor points to before the first row.
- By default, ResultSet object can be moved forward only and it is not updatable

RESULTSET INTERFACE

Sr	Methods
1.	**public boolean next():**
	is used to move the cursor to the one row next from the current position.
2.	**public boolean previous():**
	is used to move the cursor to the one row previous from the current position.
3.	**public boolean first():**
	is used to move the cursor to the first row in result set object.
4.	**public boolean last():**
	is used to move the cursor to the last row in result set object.
5.	**public boolean absolute(int row):**
	is used to move the cursor to the specified row number in the ResultSet object.

Sr	Methods
6.	**public boolean relative(int row):**
	is used to move the cursor to the relative row number in the ResultSet object, it may be positive or negative.
7.	**public int getInt(int columnIndex):**
	is used to return the data of specified column index of the current row as int.
8.	**public int getInt(String columnName):**
	is used to return the data of specified column name of the current row as int.
9.	**public String getString(String columnName):**
	is used to return the data of specified column name of the current row as String.
10	**public String getString(int columnIndex):**
	is used to return the data of specified column index of the current row as String.

RESULTSET INTERFACE

CONNECTION INTERFACE

- A Connection is the session between java application and database.
- The Connection interface is a factory of Statement, PreparedStatement, and DatabaseMetaData i.e. object of Connection can be used to get the object of Statement and DatabaseMetaData.
- The Connection interface provide many methods for transaction management like commit(), rollback() etc.
- By default, connection commits the changes after executing queries.

CONNECTION INTERFACE

Sr	Methods
1.	**public Statement createStatement():**
	creates a statement object that can be used to execute SQL queries.
2.	**public Statement createStatement(int resultSetType,int resultSetConcurrency):**
	Creates a Statement object that will generate ResultSet objects with the given type and concurrency. Forward Only (ResultSet.TYPE_FORWARD_ONLY,ResultSet.TYPE_SCROLL_INSENSITIVE,SENSITIVE, ResultSet.CONCUR_READ_ONLY,ResultSet.CONCUR_UPDATABLE
3.	**public void setAutoCommit(boolean status):**
	is used to set the commit status. By default it is true.

CONNECTION INTERFACE

Sr	Methods
4.	**public void commit():**
	saves the changes made since the previous commit/rollback permanent.
5.	**public void rollback():**
	Drops all changes made since the previous commit/rollback.
6.	**public void close():**
	closes the connection and Releases a JDBC resources immediately.

STATEMENT INTERFACE

- The Statement interface provides methods to execute queries with the database.
- The statement interface is a factory of ResultSet i.e. it provides factory method to get the object of ResultSet.

STATEMENT INTERFACE

Sr	Methods
1.	**public ResultSet executeQuery(String sql):**
	is used to execute SELECT query. It returns the object of ResultSet.
2.	**public int executeUpdate(String sql):**
	is used to execute specified query, it may be create, drop, insert, update, delete etc.
3.	**public Boolean execute(String sql):**
	is used to execute queries that may return multiple results.
4.	**public int[] executeBatch():**
	is used to execute batch of commands.

PREPAREDSTATEMENT INTERFACE

- The PreparedStatement interface is a sub interface of Statement. It is used to execute parameterized query.
- Let's see the example of parameterized query:
- String sql="insert into emp values(?,?,?)";
- We are passing parameter (?) for the values. Its value will be set by calling the setter methods of PreparedStatement.
- ? is place holder used in the Prepared statement. It is known as Parameter marker.
- Improves performance: The performance of the application will be faster if you use PreparedStatement interface because query is compiled only once.

PREPAREDSTATEMENT INTERFACE

- How to get the instance of PreparedStatement?
- The prepareStatement() method of Connection interface is used to return the object of PreparedStatement.
- Syntax:

```
public PreparedStatement prepareStatement(String query)throws SQLException{ }
```

PREPAREDSTATEMENT INTERFACE

Sr	Methods
1.	**public void setInt(int paramIndex, int value)**
	sets the integer value to the given parameter index.
2.	**public void setString(int paramIndex, String value)**
	sets the String value to the given parameter index.
3.	**public void setFloat(int paramIndex, float value)**
	sets the float value to the given parameter index.

Sr	Methods
4.	**public void setDouble(int paramIndex, double value)**
	sets the double value to the given parameter index.
5.	**public int executeUpdate()**
	executes the query. It is used for create, drop, insert, update, delete etc.
6.	**public ResultSet executeQuery()**
	executes the select query. It returns an instance of ResultSet.

PREPAREDSTATEMENT INTERFACE

PREPARED STATEMENT EXAMPLE

```
import java.sql.*;
import javax.swing.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
public class PrepStatDemo {
	static final String DB_URL = "jdbc:mysql://localhost/S";
	static final String USER = "root";
	static final String PASS = "";
	public static void main(String[] args) {
		JFrame frame = new JFrame("Insert Record
Demo");
		JLabel jlenroll = new JLabel("Enrollment No : ");
		JLabel jlname = new JLabel("Name : ");
		JLabel jladdress = new JLabel("Adderss : ");
		JLabel jlmobile = new JLabel("Mobile No : ");
		JLabel jlemail = new JLabel("Email : ");
		JLabel jlsearch = new JLabel("Search");
		JTextField jfenroll = new JTextField();
		JTextField jfname = new JTextField();
		JTextField jfaddress = new JTextField();
		JTextField jfmobile = new JTextField();
		JTextField jfemail = new JTextField();
		JTextField jfsearch = new JTextField();
		JButton jbinsert = new JButton("Insert");
		JButton jbupdate = new JButton("Update");
		JButton jbdelete = new JButton("Delete");
```

PREPARED STATEMENT EXAMPLE

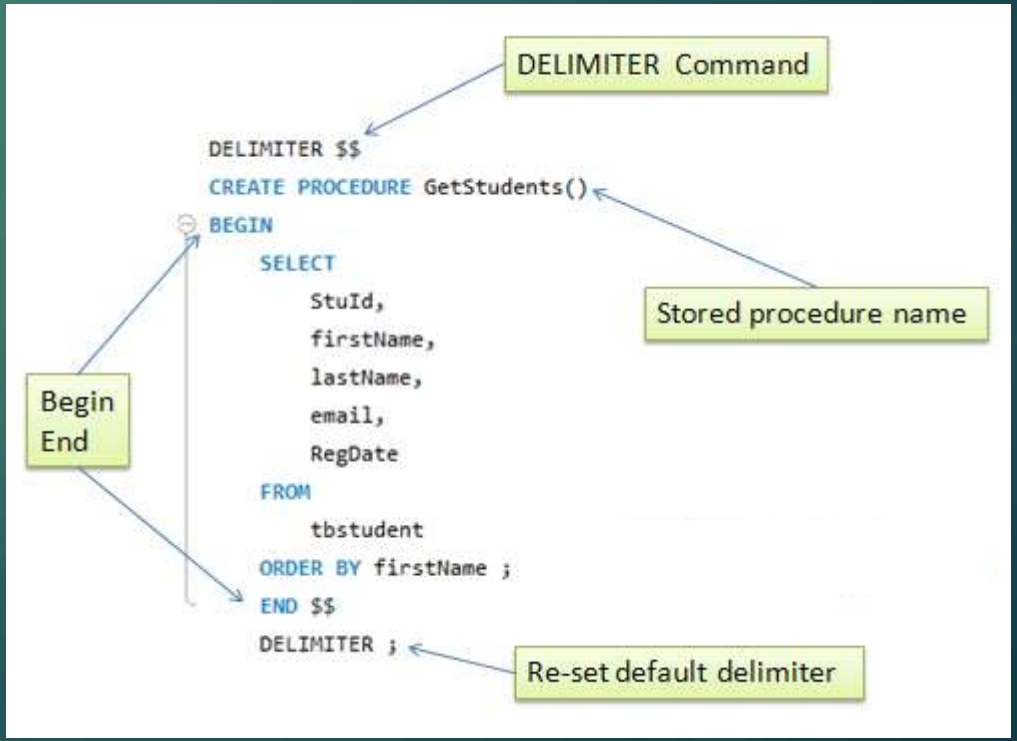
```
jlenroll.setBounds(40, 40, 100, 30);
jlname.setBounds(40, 80, 100, 30);
jladdress.setBounds(40, 120, 100, 30);
jlmobile.setBounds(40, 160, 100, 30);
jlemail.setBounds(40, 200, 100, 30);
jfenroll.setBounds(140, 40, 200, 30);
jfname.setBounds(140, 80, 200, 30);
jfaddress.setBounds(140, 120, 200, 30);
jfmobile.setBounds(140, 160, 200, 30);
jfemail.setBounds(140, 200, 200, 30);
jfsearch.setBounds(140, 350, 200, 30);
jbinsert.setBounds(140, 270, 100, 30);
jbupdate.setBounds(340, 350, 100, 30);
jbdelete.setBounds(440, 350, 100, 30);
frame.add(jlenroll);frame.add(jlname);
frame.add(jladdress);frame.add(jlmobile);
frame.add(jlemail);frame.add(jlsearch);
frame.add(jfenroll);frame.add(jfname);

frame.add(jfaddress);frame.add(jfmobile);frame.add(jbinsert);
frame.add(jfemail);

frame.add(jfsearch);frame.add(jbdelete);frame.add(jbupdate);
frame.setSize(600,500);
frame.setLayout(null);
frame.setVisible(true);
```

PREPARED STATEMENT EXAMPLE

```
jbinsert.addActionListener(new ActionListener() {
                    public void actionPerformed(ActionEvent e) {
                        int rollno = Integer.parseInt(jfenroll.getText());
                        String name = jfname.getText();
                        String address = jfaddress.getText();
                        int mobile = Integer.parseInt(jfmobile.getText());
                        String email = jfemail.getText();
                        try  {
                            Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
                PreparedStatement stmt=conn.prepareStatement("INSERT INTO Registration VALUES(?,?,?,?,?)");
                            stmt.setInt(1,rollno);//1 specifies the first parameter in the query
                            stmt.setString(2,name);
                            stmt.setString(3,address);
                            stmt.setInt(4,mobile);
                            stmt.setString(5,email);
                            stmt.executeUpdate();
                            JOptionPane.showMessageDialog(frame,"Inserted records into the table...");
                            } catch (SQLException er) {
                            er.printStackTrace();
                        }
                    }
                });
}
```

CALLABLE STATEMENT

- This interface extends Prepared statement interface and provides support for both input as well as output parameter.
- Callable statement represents the stored procedure.
- Stored procedure: It is a subroutine used by applications to access data from a database.



CALLABLE STATEMENT EXAMPLE

```
Create procedure myProcedure (IN name VARCHAR(30), IN sal INT, IN loc VARCHAR(45))
   -> BEGIN
   -> INSERT INTO Employee(Name, Salary, Location) VALUES (name, sal, loc);
   -> END //
```

```
import java.sql.CallableStatement;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
public class CallableStatementExample {
   public static void main(String args[]) throws SQLException {
      //Registering the Driver
      DriverManager.registerDriver(new com.mysql.jdbc.Driver());

      //Getting the connection
      String mysqlUrl = "jdbc:mysql://localhost/testdb";
      Connection con = DriverManager.getConnection(mysqlUrl, "root", "password");
      System.out.println("Connection established......");
      //Preparing a CallableStateement
      CallableStatement cstmt = con.prepareCall("{call myProcedure(?, ?, ?)}");

      cstmt.setString(1, "Raghav");
      cstmt.setInt(2, 3000);
      cstmt.setString(3, "Hyderabad");

      cstmt.setString(1, "Kalyan");
      cstmt.setInt(2, 4000);
      cstmt.setString(3, "Vishakhapatnam");
```

```
      cstmt.setString(1, "Rukmini");
      cstmt.setInt(2, 5000);
      cstmt.setString(3, "Delhi");

      cstmt.setString(1, "Archana");
      cstmt.setInt(2, 15000);
      cstmt.setString(3, "Mumbai");

      cstmt.execute();
      System.out.println("Rows inserted ....");
   }
}
```

Steps to connect database with java application

Steps to connect database with java application

- There are 5 steps to connect any java application with the database in java using JDBC. They are as follows:
  1. Register the driver class
  2. Creating connection
  3. Creating statement
  4. Executing queries
  5. Closing connection

STEP 1: REGISTER THE DRIVER CLASS

- The forName() method of Class class is used to register the driver class.
- This method is used to dynamically load the driver class.
- Syntax :

```
public static void forName(String className)throws
ClassNotFoundException
```

- Example to register the OracleDriver class

```
Class.forName("oracle.jdbc.driver.OracleDriver");
```

Step: 2 Create the Connection Object

- The getConnection() method of DriverManager class is used to establish connection with the database.
- Syntax:

```
public static Connection getConnection(String url) throws SQLException
```

```
public static Connection getConnection(String url,String name, String password) throws SQLException
```

- Example: To establish connection with the Oracle database:

```
Connection con=DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe", "system", "password");
```

Step: 3 Create the Statement Object:

- The createStatement() method of Connection interface is used to create statement.
- The object of statement is responsible to execute queries with the database.
- Syntax:

```
public Statement createStatement()throws SQLException
```

- Example:

```
Statement stmt=con.createStatement();
```

STEP: 4 EXECUTE THE QUERY

- The executeQuery () method of Statement interface is used to execute queries to the database.
- This method returns the object of ResultSet that can be used to get all the records of a table.
- Syntax:

```
public ResultSet executeQuery(String sql)
throws SQLException
```

- Example:

```
ResultSet rs=stmt.executeQuery("select * from emp");
while(rs.next())
{
        System.out.println(rs.getInt(1)+"
"+rs.getString(2));
}
```

Step: 5 Close the Connection Object

- The close() method of Connection interface is used to close the connection.
- By closing connection object, statement and ResultSet will be closed automatically.
- Syntax:

```
public void close()throws SQLException
```

- Example:

```
con.close();
```

JDBC DATABASE EXAMPLE

```
package swingDemo;
import java.sql.*;
public class DataBaseDemo
{
        private static void initializeDB()
        {
                Connection cn;
                Statement stmt;
                ResultSet rs;
                try {
                        Class.forName("com.mysql.jdbc.Driver");
                        cn=DriverManager.getConnection("jdbc:mysql:
                                                        //localhost:3306/
mydb","root",“pass");
                        System.out.println("Connecting to Database...!!!");
```

```
stmt=cn.createStatement();
System.out.println("===== Select Database Fetching Records ====");
System.out.println("Creating a Statement...!!!");
rs = stmt.executeQuery("Select * from stud");
System.out.println("Executing a Query...!!!");
while(rs.next())
{
System.out.print("Id ="+rs.getString("id")+"\t");
System.out.println("Name ="+rs.getString("name"));
}
```

```
        catch(Exception e){
                System.out.println(e);
                }
        }
        public static void main(String args[])
        {
                System.out.println("Welcome to Simple Database Operation...!!!");
                initializeDB();
                System.out.println("Good Bye...!!!");
        }
}
```

Problems @ Javadoc Declaration Console

```
<terminated> SelectDemo [Java Application] C:\Program Files\Java\jdk-20\bin\javaw.exe (O
Welcome to Simple Database Operation...!!!
Connecting to Database...!!!
===== Select Database Fetching Records ====
Creating a Statement...!!!
Executing a Query...!!!
Id =5   Name =sadf
Id =103 Name =Sumit
Id =111 Name =d
Id =123 Name =ds
Good Bye...!!!
```

JDBC DRIVERS
JDBC-ODBC BRIDGE

JDBC DRIVERS

- JDBC driver is a software component that enables java application to interact with database.
- It implements the defined interfaces in the JDBC API for interacting with your database server.
- For example, using JDBC drivers enable you to open database connections and to interact with it by sending SQL or database commands then receiving results with Java.
- The Java.sql package that ships with JDK contains various classes with their behaviours defined and their actual implementations are done in third-party drivers. Third party vendor's implements the java.sql.Driver interface in their database driver.

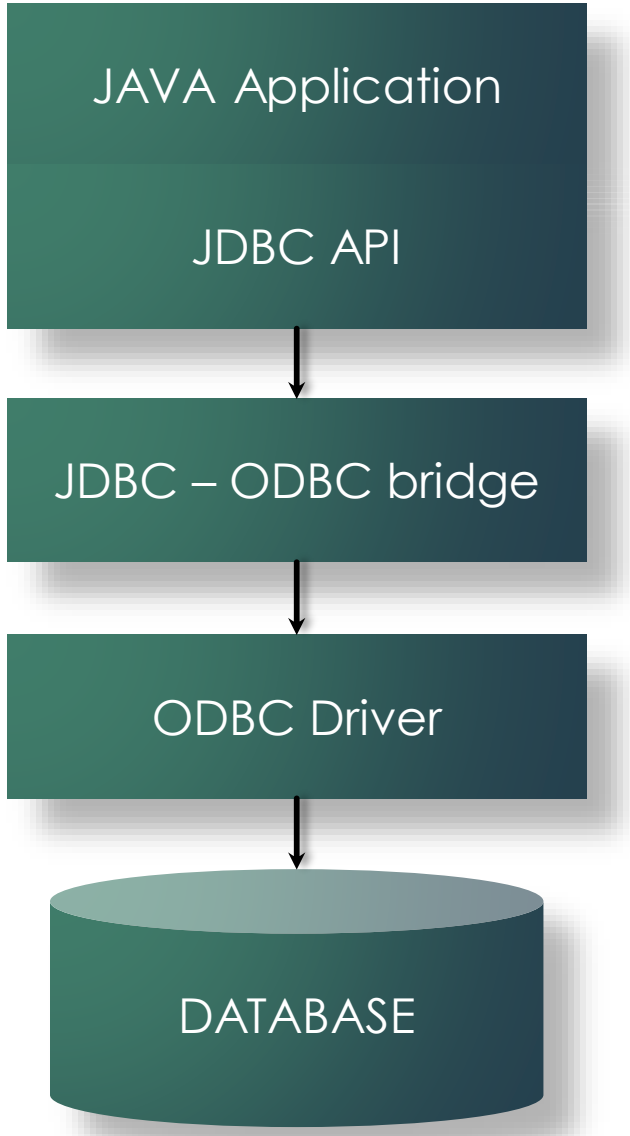
JDBC DRIVER TYPES:

- JDBC driver implementations differ because of the wide variety of operating systems and hardware platforms in which Java operates. Sun has divided the implementation types into four categories:
  - Type 1: JDBC ODBC Bridge driver (Bridge)
  - Type 2: Native-API/partly Java driver (Native)
  - Type 3: All Java/Net protocol driver(Middleware)
  - Type 4: All Java/JDBC Net Pure driver

TYPE 1: JDBC-ODBC BRIDGE DRIVER:

- In a Type 1 driver, a JDBC bridge is used to access ODBC drivers installed on each client machine.
- This driver translates JDBC method calls into ODBC function calls. The Bridge implements JDBC for any database for which an ODBC driver is available.
- When Java first came out, this was a useful driver because most databases only supported ODBC access but now this type of driver is recommended only for experimental use or when no other alternative is available.
- The JDBC-ODBC Bridge that comes with JDK 1.2 is a good example of this kind of driver.

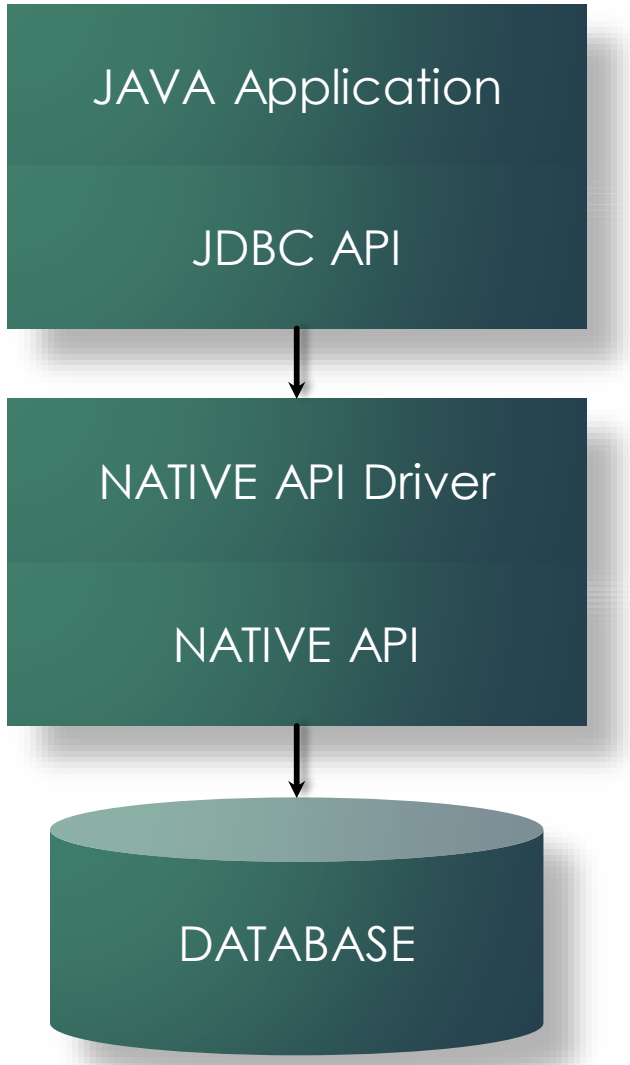
TYPE 1: JDBC-ODBC BRIDGE DRIVER:



TYPE 2: JDBC-NATIVE API:

- In a Type 2 driver, JDBC API calls are converted into native C/C++ API calls which are unique to the database.
- These drivers typically provided by the database vendors and used in the same manner as the JDBC-ODBC Bridge, the vendor-specific driver must be installed on each client machine.
- If we change the Database we have to change the native API as it is specific to a database and they are mostly obsolete now but you may realize some speed increase with a Type 2 driver, because it eliminates ODBC's overhead.
- The Oracle Call Interface (OCI) driver is an example of a Type 2 driver.

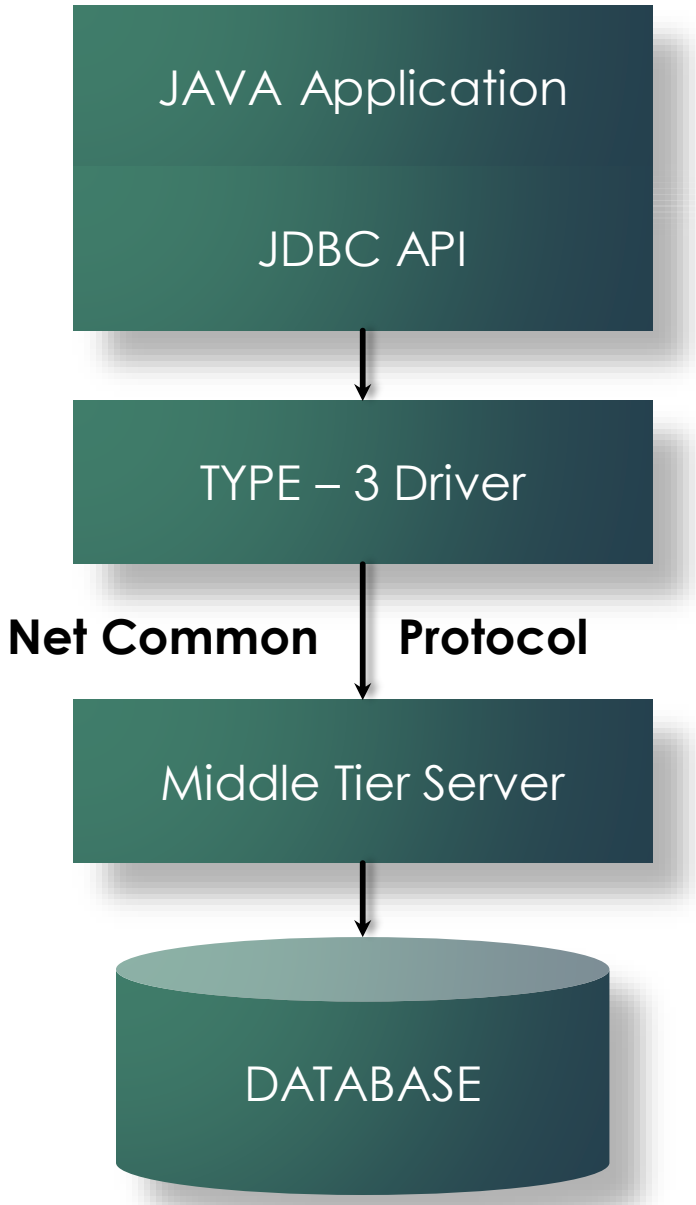
TYPE 2: JDBC-NATIVE API:



TYPE 3: JDBC-NET PURE JAVA: (NETWORK PROTOCOL DRIVER)

- In a Type 3 driver, a three-tier approach is used to accessing databases. The JDBC clients use standard network sockets to communicate with a middleware application server.
- The socket information is then translated by the middleware application server into the call format required by the DBMS, and forwarded to the database server.
- This kind of driver is extremely flexible, since it requires no code installed on the client and a single driver can actually provide access to multiple databases. We can think of the application server as a JDBC "proxy," meaning that it makes calls for the client application. As a result, we need some knowledge of the application server's configuration in order to effectively use this driver type.

TYPE 3: JDBC-NET PURE JAVA: (NETWORK PROTOCOL DRIVER)



TYPE 4: 100% PURE JAVA:

- In a Type 4 driver, a pure Java-based driver that communicates directly with vendor's database through socket connection. This is the highest performance driver available for the database and is usually provided by the vendor itself.
- This kind of driver is extremely flexible; you don't need to install special software on the client or server. Further, these drivers can be downloaded dynamically.
- MySQL's Connector/J driver is a Type 4 driver. Because of the proprietary nature of their network protocols, database vendors usually supply type 4 drivers.

JDBC- Advantages and Disadvantages

JDBC Advantages

- SQL Compatibility
- Database Vendor Support
- Can read any database if proper drivers are installed.
- Creates XML structure of data from database automatically.
- No content conversion required.
- Query and stored procedures supported.
- Can be used for both Synchronous and Asynchronous processing

JDBC Disadvantages (Limitations)

- Manual Resource Management - Connection Open, Close etc. Manually.
- Needs database specific queries.
- Lack of built-in support for Object-Relational Mapping (ORM).
- Handling and closing JDBC connections is a big issue.
- Can't maintain database independent SQL statements.
- Performance is affected if multiple connections are created and closed.
- Less Suitable for Complex Data Retrieval and Data Manipulation Task.